

Persistence (II)

File System - 3

中国人民大学信息学院

卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

Outline

- Linux I/O Stack
- File System
- File System Implementation
- Fast File System
- **FSCK and Journaling**

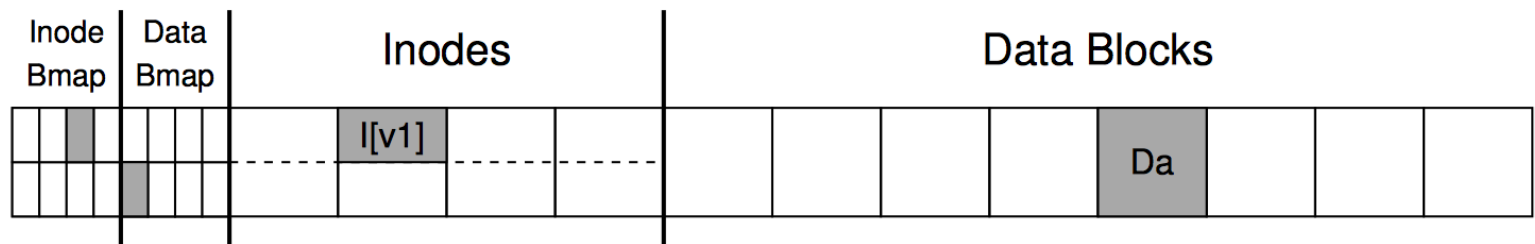
FSCK and Journaling

- 可容忍的系统状态：
 - 完全正常、完全失效（e.g, RAM）
 - 部分失效【存储/数据库、分布式系统】
- **Crash-consistency problem**
 - 例如连续2次写入高度关联的A和B，在中间发生crash
 - 原子性问题
 - 系统进入一个inconsistent状态
 - 解决方法
 - FSCK: file system checker
 - Journaling: write-ahead log

FSCK and Journaling

- 一个例子
 - 一个文件大小为1，只包括一个数据块
 - **inode**部分域:

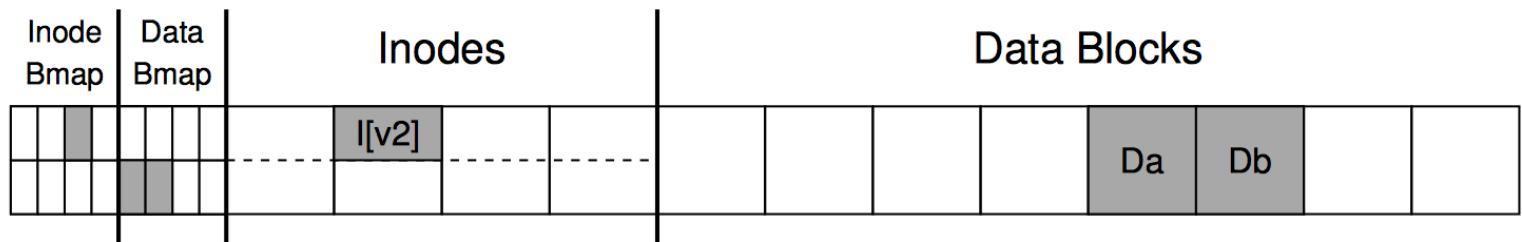
owner	: remzi
permissions	: read-write
size	: 1
pointer	: 4
pointer	: null
pointer	: null
pointer	: null



FSCK and Journaling

- 一个例子
 - 后面文件又追加写入一个block Db
 - **inode**部分域:

owner	:	remzi
permissions	:	read-write
size	:	2
pointer	:	4
pointer	:	5
pointer	:	null
pointer	:	null



FSCK and Journaling

- 三个具体的写操作
 - Bitmap
 - Inode
 - Data block (Db)
- 先写入page cache/buffer cache, 延迟写入磁盘
 - 加入中间crash, 有多种可能:

有几种出错的可能?

FSCK and Journaling

- #1. Just the data block (Db) is written to disk
 - db内容丢失(lost update), 但不会造成其他问题
- #2. Just the updated inode (I[v2]) is written to disk
 - inode与bitmap不一致, 且inode指向dirty db
- #3. Just the updated bitmap (B[v2]) is written to disk
 - inode与bitmap不一致, 造成space leak

FSCK and Journaling

- #4. inode ($I[v2]$) and bitmap ($B[v2]$) written, but not data (Db)
 - 元数据一致，但是数据是垃圾数据
- #5. inode ($I[v2]$) and data block (Db) written, but not the bitmap ($B[v2]$)
 - 读取正确，但数据可能被覆盖
- #6. bitmap ($B[v2]$) and data block (Db) written, but not the inode ($I[v2]$)
 - 元数据不一致

FSCK and Journaling

- Solution #1: The File System Checker
 - FSCK最大的问题：
 - 整盘扫描，太慢了！
 - 是一种补救措施，只能修复部分问题

FSCK and Journaling

- Solution #2: Journaling (Write-Ahead Logging)
 - WAL来自于database领域
 - 最早使用journaling的文件系统是Cedar(1987)
 - 现在广范使用: Linux ext3, ext4, reiserfs, JFS, XFS, NTFS

journal(ing) == log(ging)

有时候不同领域用同一个词描述不同的东西

有时候不同领域同不同的词描述同一个东西

FSCK and Journaling

- Solution #2: Journaling (Write-Ahead Logging)
 - 将上层用户对FS的操作看做一个transaction (tx)
 - 例如向某文件中追加一个data block (db)
 - 一个tx中可能包含多个对FS内部数据的写入
 - 例如写入inode(I[v2]), bitmap(B[v2]), 和Db
 - 在写数据前, 先写log, 描述要做什么

FSCK and Journaling

- Solution #2: Journaling (Write-Ahead Logging)
 - 当写log完成时，即可告知用户操作完成
 - 如果写log过程中系统崩溃，恢复时则无视可能不完整的log，就当tx没发生
 - 如果写数据过程中系统崩溃，恢复时可以依据log的内容继续完成数据写入

FSCK and Journaling

- Solution #2: Journaling (Write-Ahead Logging)
 - 如果某个时间点之前的全部tx都已经完成对FS的修改，则清除该时间点之前的log并生成checkpoint
 - 避免log无限增长
 - 每次恢复时，只需要扫描日志区中最近一次checkpoint之后的log blocks，无需扫描全盘

代价：写两遍数据；收益：快速且安全的恢复

FSCK and Journaling

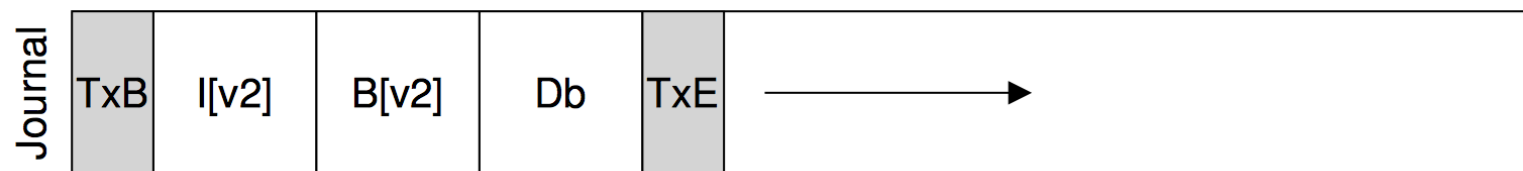
- Linux ext3的例子
 - 分成很多独立的group，每个group都有bitmap, inode, data block
 - 增加日志区域，或者放在一个独立的磁盘上



FSCK and Journaling

- Linux ext3的例子

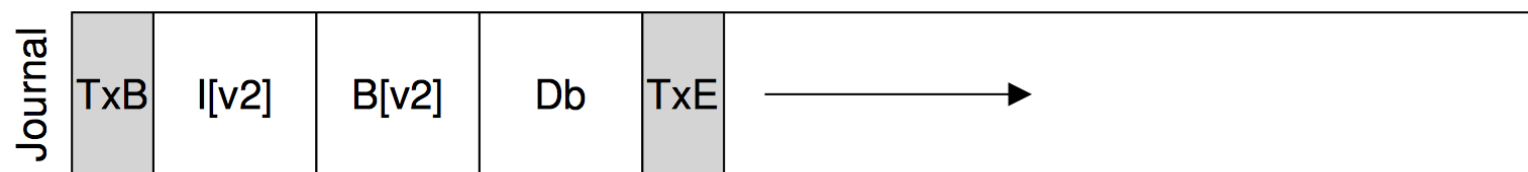
- 例如向文件中写入一个新数据块的操作包含对FS的三次写入：
 - inode(I[v2]), bitmap(B[v2]), data block (Db)
- 先将这些写操作记录到日志中：



- TxB和TxE标记txn的开始和结束
- 日志中所有的记录都会包括TID，以避免不同事务的记录发生混淆

FSCK and Journaling

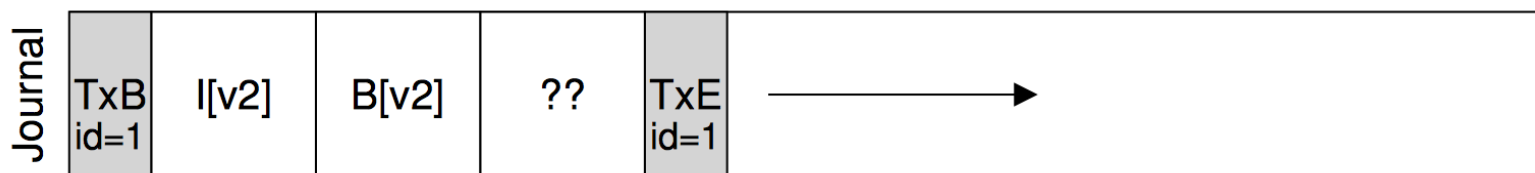
- Linux ext3的例子
 - 上页的例子中，log总共写了5个数据块，TxB, I[v2], B[v2], Db, TxE



- 写完log后即可向上层返回写入成果，稍后再进行checkpoint、真正完成对FS中bitmap、inode和data block的修改

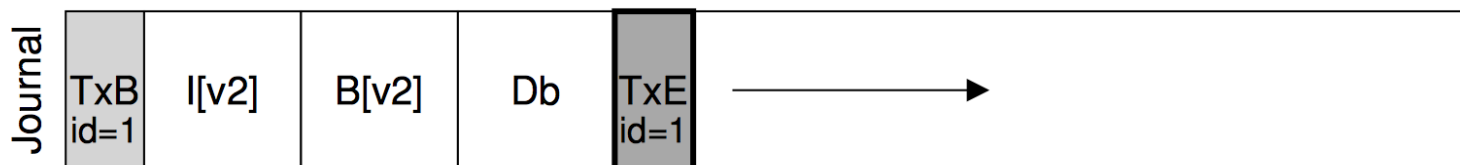
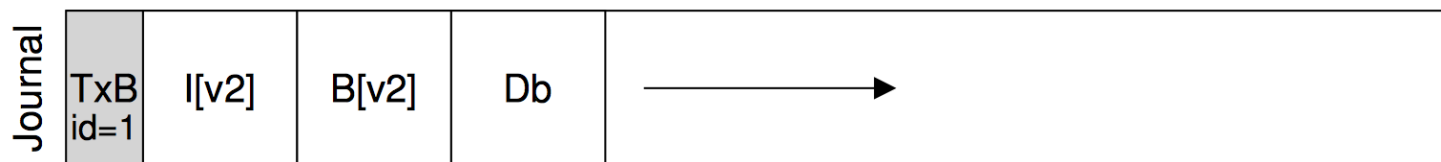
FSCK and Journaling

- 5个log block怎么写入?
 - 逐个写入? 过慢
 - 一起写? 不安全
 - block device可能乱序写入, 导致TxE落盘了, 但是之前的log block还没落盘
 - 恢复时会把??当做真正的数据内容去恢复



FSCK and Journaling

- 改进：除了TxE之外一起写，TxE最后单独写



- 存储设备保证sector/block的写入是原子性的，要么写成功，要么没写；所以TxE要小于512B且sector align

FSCK and Journaling

- 总结: Journaling (WAL) 分三步
 - Journal Write (将TxB和对metadata、data的修改写入log)
 - Journal Commit (将TxE写入log)
 - Checkpoint (完成对metadata和data的最终修改, 清理log)
- 进一步改进提升性能:
 - 对log增加checksum
 - 这样一个TxE可以和其他log block可以一起写, 省掉了第二步
 - Linux ext4采用

Tips: 以算代存, 以存代算, 都是常见的优化技巧
要具体看哪个收益大、代价小

系统设计基本思想

- Hybrid
 - 一个问题存在互补方案，则混合, e.g., Hybrid Mapping
- Abstract and Unify
 - 分层抽象，对上提供统一接口, e.g., Linux I/O stack
- Trade-off
 - 两害相权取其轻 (lesser of two evils)
 - 以时间换空间, e.g., RAID 5, Journaling checksum
 - 以顺序换随机, e.g., Buffering, Log-structured FS
 - 以延迟换吞吐, e.g., group commit (稍后讲到)

还有很多:

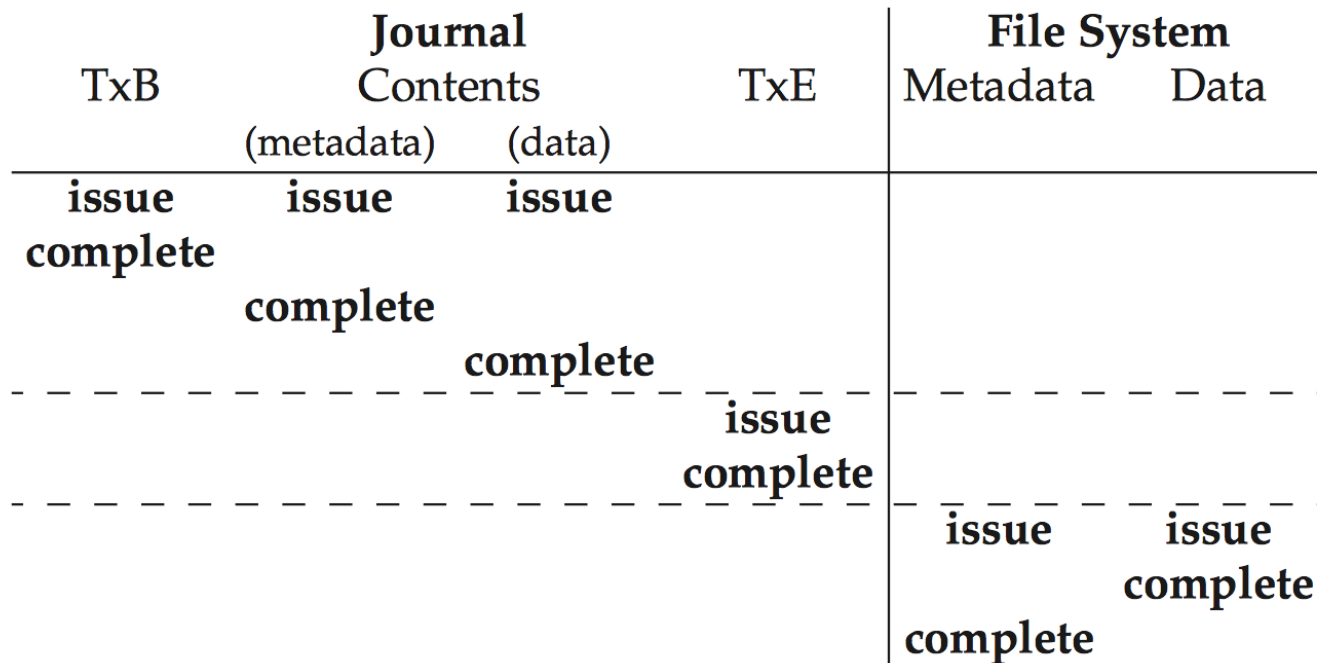
Priority, Adaptive, Divide-and-conquer, Epoch, Lineage, asynchronize...

FSCK and Journaling

- Recovery
 - 如果commit完成前crash
 - skip这些update
 - 如果commit完成后、checkpoint完成前crash
 - 则从TxB开始replay这些updates (redo log)
 - 如果checkpoint完成后crash
 - No one cares:)

FSCK and Journaling

- Journaling timeline

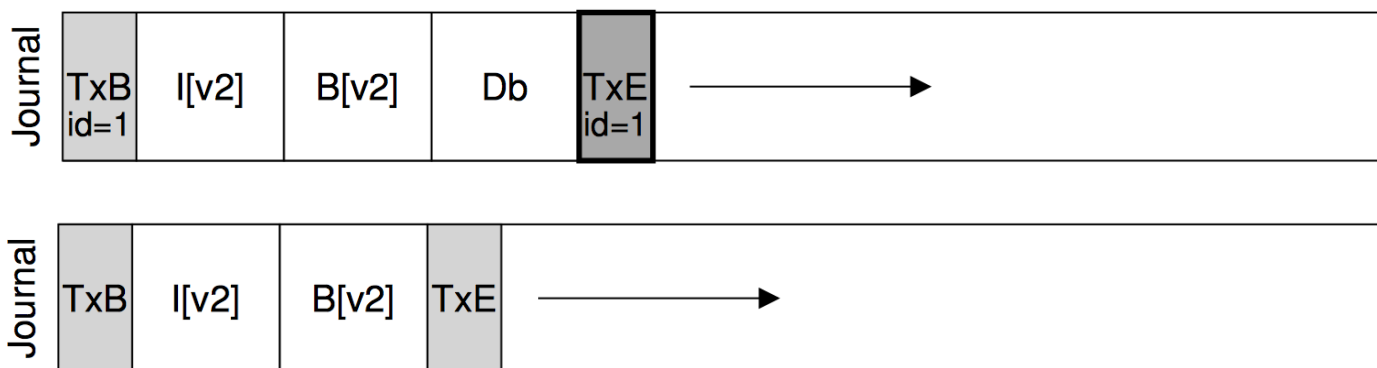


FSCK and Journaling

- 如何减少写日志的磁盘开销
 - 例如写入同一个目录的两个文件file1和file2
 - 要多次更新bitmap, 父目录的data
 - 按照上述log protocol, I/O次数很多
 - 如何优化?
 - batch commit (i.e., batching log updates)
 - 将近段时间内的tx看做一个large tx or global tx, 批量执行
 - 批量化的思想在CS中非常常见, 本质是牺牲latency换取throughput

FSCK and Journaling

- Metadata Journaling / Ordered Journaling
 - 目前方案的问题：两倍I/O traffic
 - 解决方案：
 - 日志中去掉data block，减少绝大多数I/O
 - Db直接写到原位置，避免重复写



Db什么时候写入？

FSCK and Journaling

- Metadata Journaling / Ordered Journaling
 - Db什么时候写入？
 - 在TxE之后，不行：恢复时可能没有正确Db
 - 在日志之前：
 - 1. Data Write: in-place write to final location
 - 2. Journal Metadata Write
 - 3. Journal Commit, wait for 1 and 2 to complete
 - 4. Checkpoint Metadata and free (释放日志空间)

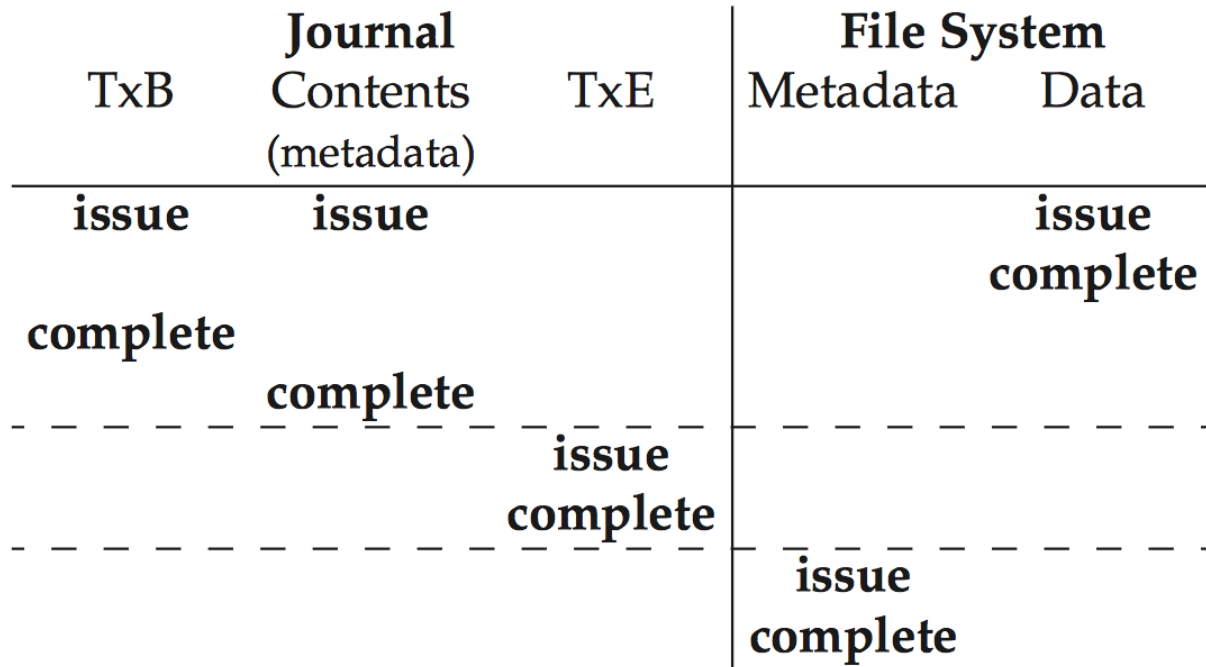
避免在log block和data block中两次写入用户数据

由于用户数据通常远大于metadata，这样可以节省大量I/O

1和2的顺序可以交换吗？

FSCK and Journaling

- Metadata journaling timeline

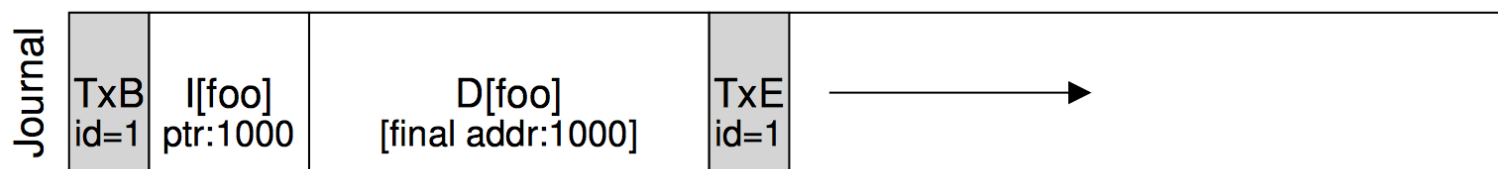


FSCK and Journaling

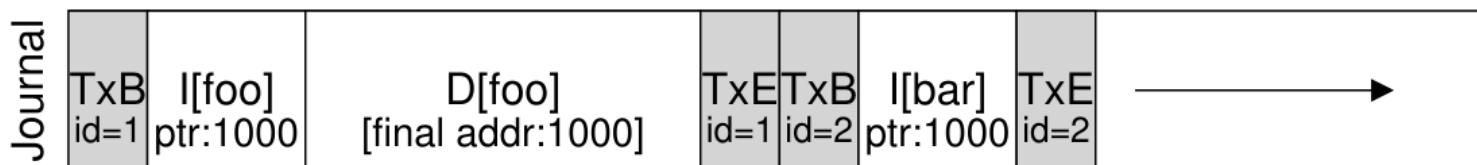
- Metadata Journaling Applications:
 - Windows NTFS, SGI's XFS
 - Ext3可选ordered journaling mode, 等价于 metadata journaling

FSCCK and Journaling

- Metadata Journaling的bug:
 - 假设创建了一个目录foo，其data block为1000
 - 目录的data block被认为是元数据，所以写入日志



- 然后删除foo目录，然后写入一个新的文件bar，刚好reuse data block 1000



会有什么问题？

FSCK and Journaling

- Metadata Journaling的bug:
 - bar文件的data block会直接写到data block 1000，不在log中记录
 - 但是当恢复时，会先把目录foo的data block写到1000，覆盖bar文件的内容

这个问题的根本原因是什么？
如何解决？

FSCK and Journaling

- Metadata Journaling的bug:
 - ext3的解决方案:
 - 在日志中增加revoke（删除）record
 - 恢复时先扫描日志，如果有revoke，revoked data（foo的data block 1000）不会被恢复

为什么不把directory的data block当成data?

FSCK and Journaling

- Solution #3: Other Approaches
 - Soft Updates
 - 精心设计FS的内部结构和写入顺序
 - 例如先写data block再写indirect nodes, 最后再写inode可以避免inode指向dirty data blocks
 - 实现比较复杂

FSCK and Journaling

- Solution #3: Other Approaches
 - Copy-On-Write (COW)
 - 用于Log-structured FS, ZFS
 - 永远不原位更新，而是写到新的位置上
 - 本质上是一种多版本（multi-versioning）的实现

回想一下Flash SSD中的hybrid mapping?

FSCK and Journaling

- Solution #3: Other Approaches
 - Backpointer-Based Consistency (BBC)
 - 数据块中增加back pointer，指向inode
 - 通过判断forward pointer和back pointer是否一致，来判断inode和data block是否一致
 - back pointer起到对parent的校验作用

是不是有点区块链的味道？

F2FS (Flash-friendly File System)

- 三星最早开发，华为、Google等参与开发
- 也是基于Copy-on-write (COW) 思想
- 应用于安卓设备和部分Linux服务器

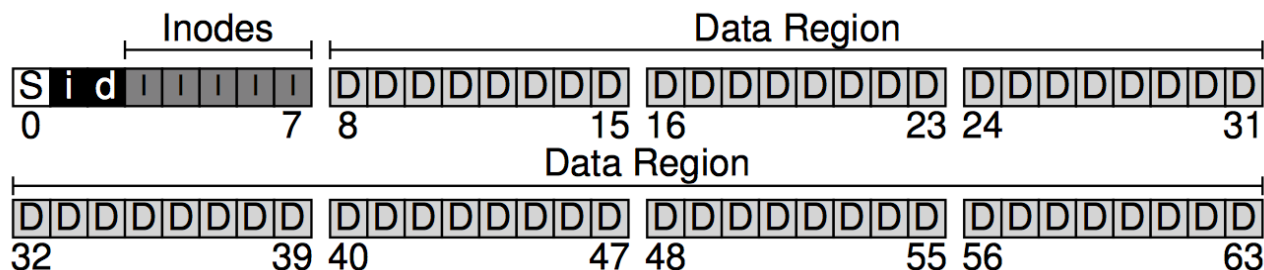
F2FS: A New File System for Flash Storage

Changman Lee, Dongho Sim, Joo-Young Hwang, and Sangyeun Cho

<https://www.usenix.org/conference/fast15/technical-sessions/presentation/lee>

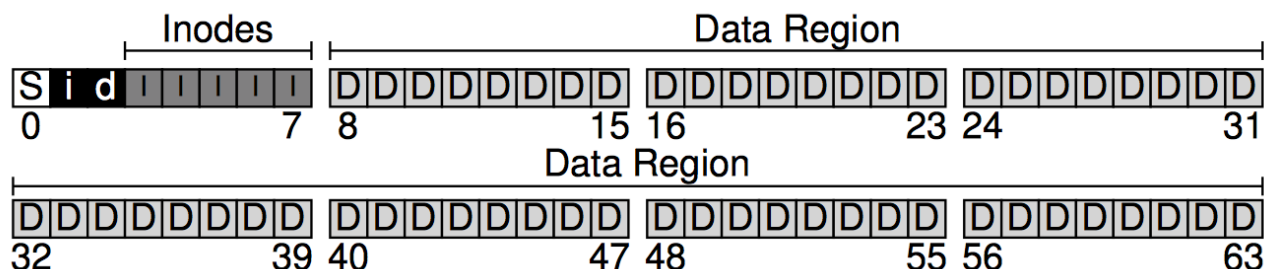
课堂练习

- 一个文件系统的数据分布类似VSFS, inode包括15个指针（12个direct, 2个indirect, 1个double indirect）写入一个10MB的/foo/bar文件，分别采用data journaling和metadata journaling，分别需要多少次I/O？（除了文件data block外的都算元数据，每个4KB block的读或写都算一次I/O，创建或写入同一文件只需要最终保存一次metadata，向log中写一个record也算写入一次I/O）



课堂练习（答案）

- 一个文件系统的数据分布类似VSFS, inode包括15个指针（12个direct, 2个indirect, 1个double indirect）写入一个10MB的/foo/bar文件，分别采用data journaling和metadata journaling，分别需要多少次I/O？（除了文件data block外的都算元数据，每个4KB block的读或写都算一次I/O，创建或写入同一文件只需要最终保存一次metadata，向log中写一个record也算写入一次I/O）

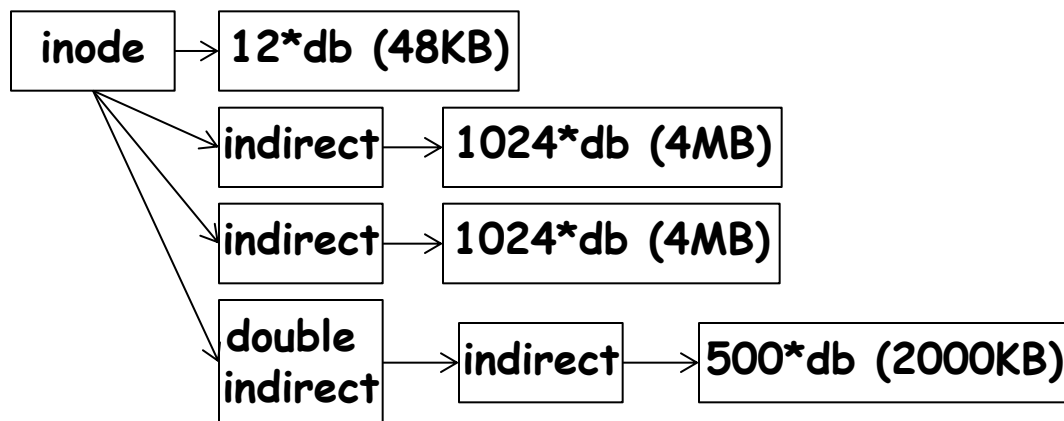


首先，读取superblock，读取/的inode和data block，读取/foo的inode和data block。为了给/foo/bar创建inode和data blocks，还需要读取inode bitmap和data bitmap。所以共需要7次read来读取以上的metadata

课堂练习（答案）

- 一个文件系统的数据分布类似VSFS, inode包括15个指针（12个direct, 2个indirect, 1个double indirect）写入一个10MB的/foo/bar文件，分别采用data journaling和metadata journaling，分别需要多少次I/O？（除了文件data block外的都算元数据，每个4KB block的读或写都算一次I/O，创建或写入同一文件只需要最终保存一次metadata，向log中写一个record也算写入一次I/O）

然后需要为/foo/bar创建1 inode、3 indirect pointer blocks、1 double indirect pointer block、2560 data blocks，这需要写入inode bitmap、data bitmap、1个inode、4个pointer blocks、2560个data blocks。再加上修改/foo的data block和inode，共需要9次metadata写入（inode bitmap、data bitmap、/foo/bar inode、/foo inode、/foo inode、4个pointer blocks）和2560次data写入



课堂练习（答案）

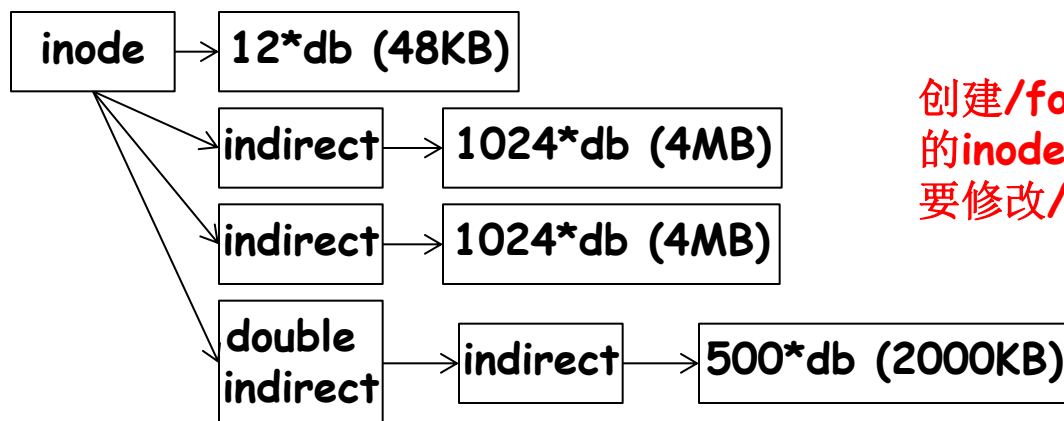
- 一个文件系统的数据分布类似VSFS, inode包括15个指针（12个direct, 2个indirect, 1个double indirect）写入一个10MB的/foo/bar文件，分别采用data journaling和metadata journaling，分别需要多少次I/O？（除了文件data block外的都算元数据，每个4KB block的读或写都算一次I/O，创建或写入同一文件只需要最终保存一次metadata，向log中写一个record也算写入一次I/O）

TxB、TxE以及metadata的修改都要写log，datajournaling和metadata journaling的区别在于2560个data blocks是否写log。综上：

读取metadata需要7次I/O

创建并写入/foo/bar以及修改父目录/foo共需要2569次I/O

对于data journaling，还需要额外2569+2次I/O来写log；对于metadata journaling，还需要额外9+2次I/O来写log



创建/foo/bar只需要修改/foo的inode和data block，不需要修改/的inode和data block

IO-FS复习

